

W-Congestion-EQ: Verifiable Congestion Budgets Proof-Carrying Certificates and Transparency Receipts for Anonymous Overlays

Draft v0.41 (2026-02-24)

Abstract

Congestion-EQ budgets are meaningful only if they are verifiable by parties other than the service operator. We present a practical assurance layer for anonymous overlays (and Anonymous-DHT lookup services) that publishes compact *congestion budget certificates* binding: (i) the declared congestion witness family, (ii) the PSC-Q (or other) compiler knobs that implement it, and (iii) the accountant arithmetic that yields an end-to-end budget. To reduce trust in a single publisher, certificates are anchored in append-only transparency logs and distributed as *receipts* with inclusion/consistency proofs.

The contribution of this paper is the verification surface and its threat model: what is mechanically checkable offline, what policies reduce equivocation (split-world), and how to connect congestion certificates to release-time change control. We keep this paper brief by importing generic receipt/log semantics from the Release-Property paper and focusing on congestion-specific certificate fields, verifiers, and deployment policies.

Scope. We define the *assurance layer* for congestion budgets: certificate fields, verifier checks, and anti-equivocation publication policies for Congestion-EQ style claims.

Units. Budgets are published in bits (use $\log_2$ and 2^ϵ); results stated in nats can be converted by dividing by $\ln 2$ (see Synthesis 8).[13]

Imports. We import the congestion witness family from Congestion-EQ and mechanisms from PSC-Q.[1, 2] Generic receipt/log semantics and release-time gating are treated as owned by the Release-Property workflow.[3] Packaging and versioning are treated as owned by ABOM/OINL and disagreement-gated recertification.[4, 5]

Exports. A *W-Congestion-EQ receipt* schema: signed epoch-level congestion budgets binding (i) the witness family, (ii) declared knob sets, and (iii) accountant arithmetic; optionally anchored into transparency logs for split-world detection.

Artifact policy. Source-only; schemas, verifiers, and microstudy plots referenced below are available on request.[6]

1 Position in the series

Congestion-EQ defines the *witness* and *budget metric* (TV constraints over congestion observables) [1]. PSC-Q defines a compiler-friendly mechanism family that meets these budgets under declared calendars and substitution padding [2]. This paper (W-Congestion-EQ) defines the *assurance layer*: how a deployment turns “we run PSC-Q with knob set θ ” into a checkable, replayable statement that third parties can verify.

We treat receipt formats, transparency logs, and release-time gating as imported infrastructure from the release-property workflow [3]. ABOM/OINL is the packaging companion: a certificate should point to an ABOM entry identifying its schema versions and evidence digests [4]. Change control (what happens when you update knobs) is governed by disagreement-gated recertification [5].

2 Threat model and goals

Participants. A provider runs an anonymity overlay and publishes periodic (e.g., daily) congestion-budget claims. Users/auditors consume those claims. **Adversary.** We consider a provider who may (i) misreport knobs or accountant outputs, (ii) publish evidence selectively, or (iii) equivocate by presenting inconsistent certificates to different audiences. **Goal.** Define a certificate/receipt interface with an offline verifier that: (1) checks internal arithmetic consistency; (2) binds claims to declared knobs and witness families; and (3) reduces equivocation via transparency receipts.

Non-goals. A passing certificate is not a proof of real-world anonymity: it does not prevent hidden side channels. It is a verifiable statement about what was claimed, how it was computed, and whether the claim is consistent with the public contract objects of the series.

3 Contract objects and what must be bound

3.1 Witness binding (Congestion-EQ import)

Congestion-EQ represents congestion leakage through a witness W (queueing delay, RTT, loss/ECN marks, or derived aggregates), and budgets the TV distance between witness distributions under different destinations/queries [1]. A congestion certificate must bind:

- the witness family identifier (including units and sampling rule);
- the policy for truncation/windowing (to avoid unbounded-support pathologies);
- the budget statement (per epoch and composed) including any reset/flush model.

3.2 Knob binding (PSC-Q import)

PSC-Q implements congestion equalization with public calendars, substitution padding, and (optionally) declared dithers [2]. A congestion certificate must bind the deployed knob set θ , including:

- calendar structure (period, slotting, any tiered calendars);
- substitution padding policy (cover distribution; feasibility constraints);
- retry discipline relevant to congestion observables (schedule-only retries, if any).

3.3 Packaging hooks (ABOM/OINL import)

We treat the certificate as an ABOM-referenced artifact:

- **Schema pinning.** The certificate names a schema version and canonicalization profile.
- **Digest binding.** The certificate carries content hashes for any evidence objects referenced (e.g., topology snapshot digest, calibration profile digest) even if those objects are not published in the source-only archive.

The point is not to publish everything, but to let a verifier say “I checked the claim under this exact set of inputs”.

4 Certificate format (congestion-specific core)

We model certificates as canonicalized JSON objects, signed, and optionally anchored in a transparency log. Generic JSON canonicalization and receipt semantics are imported from [3]; here we only specify the congestion-specific *fields and checks*.

4.1 Minimal field set

A minimal congestion certificate contains:

- **id:** deployment identifier and epoch range;
- **witness:** witness family id, sampling and truncation parameters;
- **mechanism:** PSC-Q knob manifest θ (calendar + padding);
- **accountant:** per-epoch budget terms and composed ε (or (ε, δ) if used);
- **evidence_digests:** hashes for calibration profiles and reference data;
- **signatures:** signer id and signature over canonical bytes;
- **receipts** (optional): transparency log receipts for the signed bytes.

4.2 Canonicalization

JSON has many syntactic representations of the same object. Therefore signing must be done over a canonical byte representation (e.g., RFC 8785/JCS) [7]. This is a **MUST** check: a verifier recomputes canonical bytes and verifies the signature over those bytes.

5 Mechanical verification checklist

We separate what a verifier can *mechanically* check from what is an ecosystem policy choice.

5.1 MUST checks

1. **Schema and canonicalization.** Validate the JSON object against a pinned schema version; canonicalize using the declared profile (e.g., JCS) [7].
2. **Signature.** Verify the signature over canonical bytes; verify key identity according to the trust model used by the deployment (e.g., pinned keys, or an identity framework).
3. **Arithmetic.** Recompute the accountant from the declared per-epoch terms and repetition/reset rules; check that the composed bound matches the certificate.
4. **Binding.** Check that the witness id, knob manifest, and evidence digests are present and well-formed.

5.2 SHOULD checks (anti-equivocation)

If transparency receipts are present:

1. **Inclusion.** Verify inclusion proof for the certificate hash against the signed log checkpoint.
2. **Consistency.** Verify consistency proofs between checkpoints to ensure an append-only view (monotone tree size) [8].
3. **Witness policy.** Require a witness/monitor policy (or cross-logging) that makes split-world attacks detectable at acceptable probability.

The mechanics follow Certificate Transparency v2-style receipts [8] or generic receipt formats (e.g., COSE receipts) [12].

6 Split-world risk and a simple detectability law

Transparency logs reduce equivocation only if clients/auditors obtain *comparable views*. A split-world operator could show different certificates to different audiences while maintaining two log views. CT-style defenses rely on gossip, cross-logging, and monitors [8].

We use a minimal probabilistic model to guide policy choices. Assume the operator equivocates on a fraction ρ of epochs (publishing inconsistent certificates). Let m independent witnesses each sample s epochs uniformly at random and gossip checkpoints pairwise. Under a standard “collision” approximation, the probability that at least one witness observes an equivocated epoch is

$$\Pr[\text{detect}] \approx 1 - (1 - \rho)^{ms}. \quad (1)$$

This ignores timing/correlation and is optimistic, but it gives a sizing rule: to make $\Pr[\text{detect}] \geq 1 - \eta$, it suffices that $ms \gtrsim \frac{\log_2(1/\eta)}{\rho}$. In practice, witnesses should bias sampling toward high-risk periods (software updates, key rotations) and enforce monotone-checkpoint rules.

7 Connecting to release-time change control

Congestion certificates are not static: knobs and deployments change. We recommend treating each certificate as a *release artifact* governed by the release-property workflow [3]:

- publish certificates through a policy gate that checks MUST conditions;

- anchor releases in a claims ledger and map them to ABOM/OINL entries [4];
- require recertification policies keyed to disagreement thresholds [5].

This keeps “privacy budgets” from becoming marketing text: they become attestations with change control.

8 Discussion and limitations

What this does not prevent. A provider can satisfy certificate arithmetic while leaking through channels not modeled in the witness family. The right interpretation is: “the published budget is consistent with the public contract objects.”

Why this still helps. It turns a privacy claim into an object that can be audited, diffed, and monitored over time (like SBOM-style supply-chain artifacts, but for anonymity budgets) [9, 10, 11].

9 Takeaways

- **W-Congestion-EQ is verifiability:** it turns congestion budgets into certificates that third parties can check without trusting the compiler.
- **Receipts beat narratives:** publish a verifier report plus inclusion/consistency proofs in an append-only log to reduce equivocation and enable monitoring.
- **Interfaces are small:** the certificate binds (i) a concrete plan/manifest and (ii) a dual witness that upper-bounds the congestion objective under the Congestion-EQ accountant.

References

- [1] Anonymous. *Congestion-EQ: Total-Variation Budgets for Congestion Witnesses in Anonymous Overlays*. Congestion series Paper 1, The-One-True-Archive (2026).
- [2] Anonymous. *PSC-Q: Public Service Calendars and Substitution Padding for Congestion-Equalized Anonymous Lookups*. Congestion series Paper 2, The-One-True-Archive (2026).
- [3] Anonymous. *Privacy as a Release Property: Proof-Carrying Anonymity Receipts for Anonymous DHTs*. Release & destination Paper A, The-One-True-Archive (2026).
- [4] Anonymous. *ABOM and OINL: An Anonymity Bill of Materials and an Odds-Inflation Nutrition Label*. Anonymity series Paper C, The-One-True-Archive (2026).
- [5] Anonymous. *Disagreement-Gated Recertification: Change Control for Certified Anonymous Systems*. Certified series Paper C, The-One-True-Archive (2026).
- [6] Anonymous. *Artifact and Disclosure Policy for the One-True-Archive (Source-Only Public Release)*. Synthesis 6, The-One-True-Archive (2026).
- [7] A. Rundgren. RFC 8785: JSON Canonicalization Scheme (JCS). IETF, 2020.
- [8] B. Laurie, E. Kasper, E. Messeri, and R. Stradling. RFC 9162: Certificate Transparency Version 2.0. IETF, 2021.

- [9] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappelis. in-toto: Providing farm-to-table guarantees for bits and bytes. In *USENIX Security*, 2019.
- [10] Z. Newman, J. Speed Meyers, and S. Torres-Arias. Sigstore: Software signing for everybody. In *ACM CCS*, 2022. DOI: 10.1145/3548606.3560596.
- [11] SLSA contributors. *SLSA Specification v1.0*, 2023.
- [12] IETF COSE Working Group. *COSE Receipts (Internet-Draft)*, 2025.
- [13] Anonymous. Notation and Minimal Model for the One-True-Archive. Series synthesis note (Synthesis 8), 2026. (This archive.)